

可靠传输原理

计算机网络—传输层



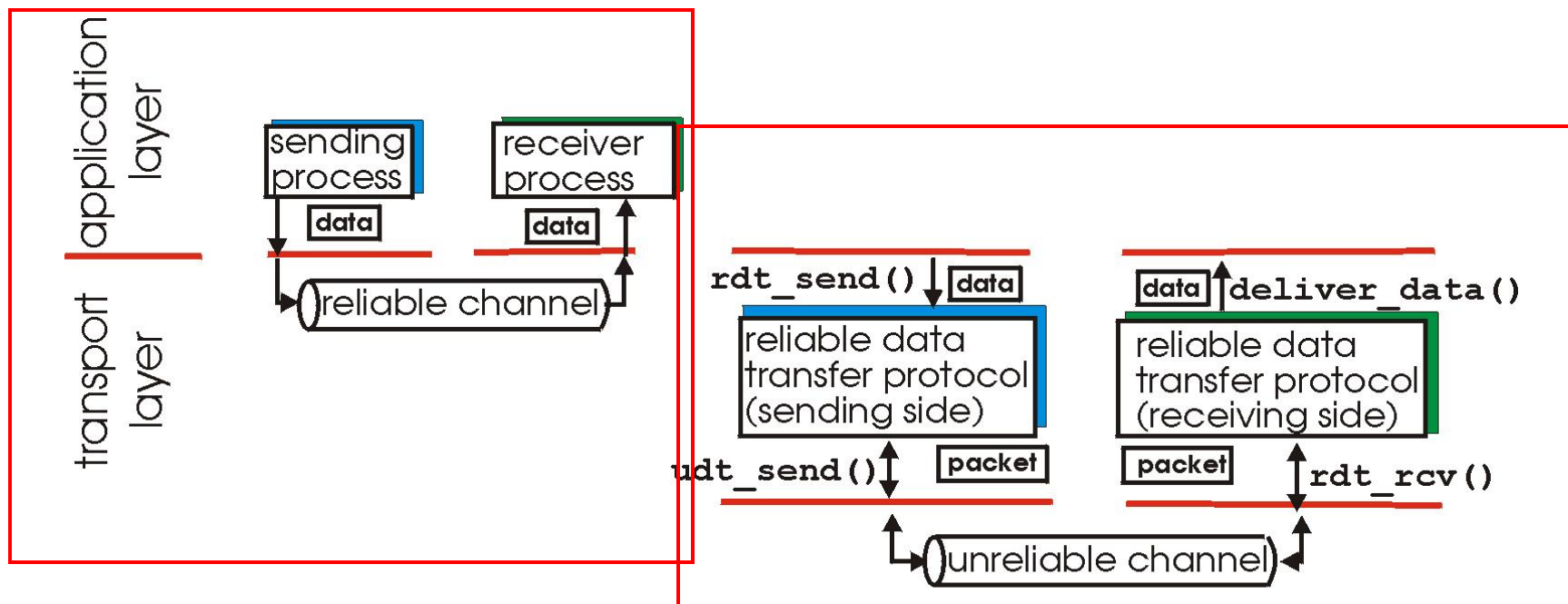
可靠数据传输的原理

- 在应用层、传输层、数据链路层的重要性
- 可靠数据传输提供给上层实体的服务抽象是，数据可以通过可靠信道进行传输（尽管再下层信道是不可靠的）
 - 数据不损坏
 - 数据丢失
 - 顺序传送
 - 刚好是TCP所提供的服务模型
- 实现这种服务抽象是可靠数据传输协议的责任，再下层不可靠信道的特点决定了本层可靠数据传输协议 (rdt) 的复杂性



可靠数据传输的原理

- 可靠数据传输原理的基本概念



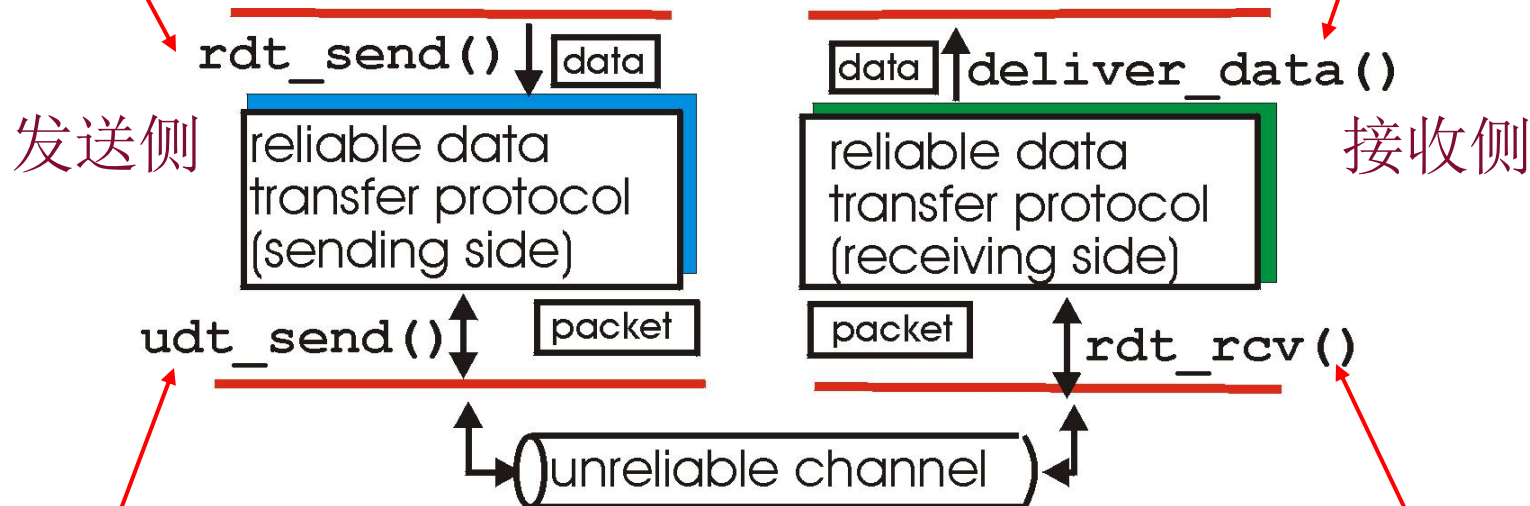
(a) provided service

(b) service implementation

可靠数据传输: 基本概念

rdt_send() :
上层应用调用
将数据发送至接收方应用层

deliver_data() :
接收到数据时调用, 发送至应用层



udt_send() : rdt调用
通过不可靠信道发送数据包到接收方

rdt_rcv() :
从信道收到数据包时调用 (触发)

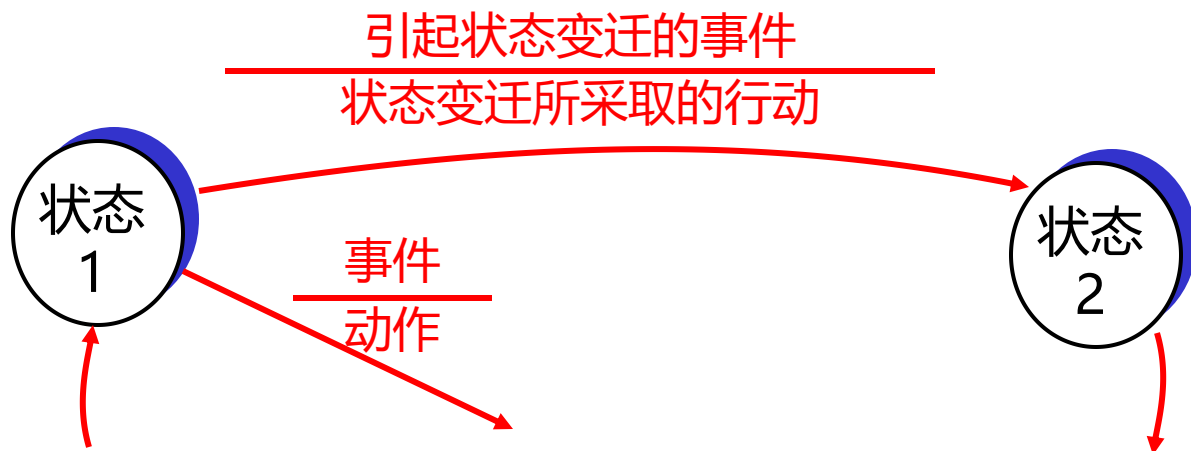


可靠数据传输: 基本概念

- 我们将:
- 仅考虑单向数据传输
- 但协议还是需要在两个方向发送分组，控制信息将在两个方向流动！
- 使用有限状态机 (FSM) 来定义发送方和接收方

状态:

当位于这个“状态时”，下个状态唯一地由下个事件决定

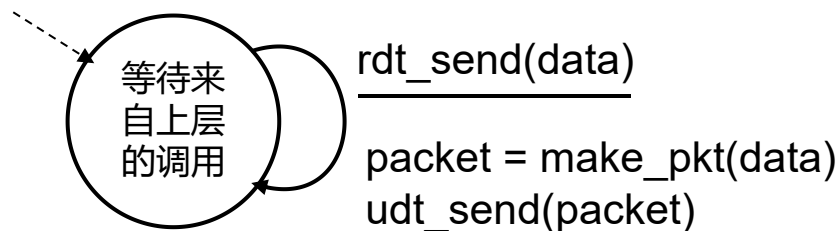
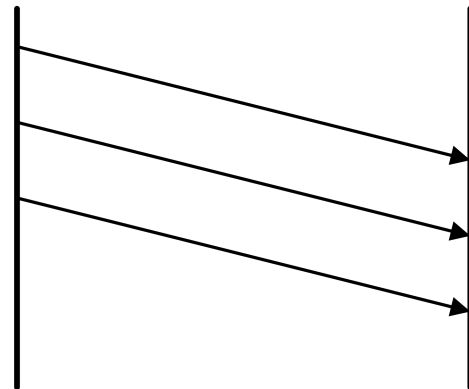


rdt1.0: 经可靠信道的可靠传输

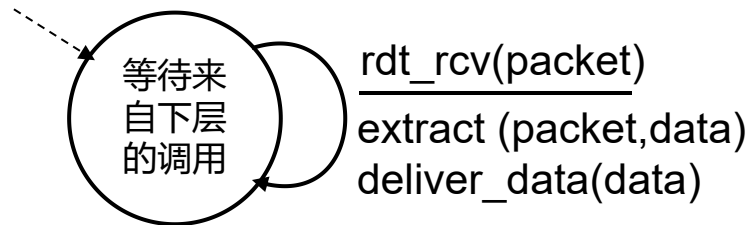
- 底层信道非常可靠
 - 无比特差错
 - 无分组丢失
 - 顺序传送
- 发送方、接收方的单独FSM:
 - 发送方将数据发向底层信道
 - 接收方从底层信道读取数据

发送方

接收方



发送方



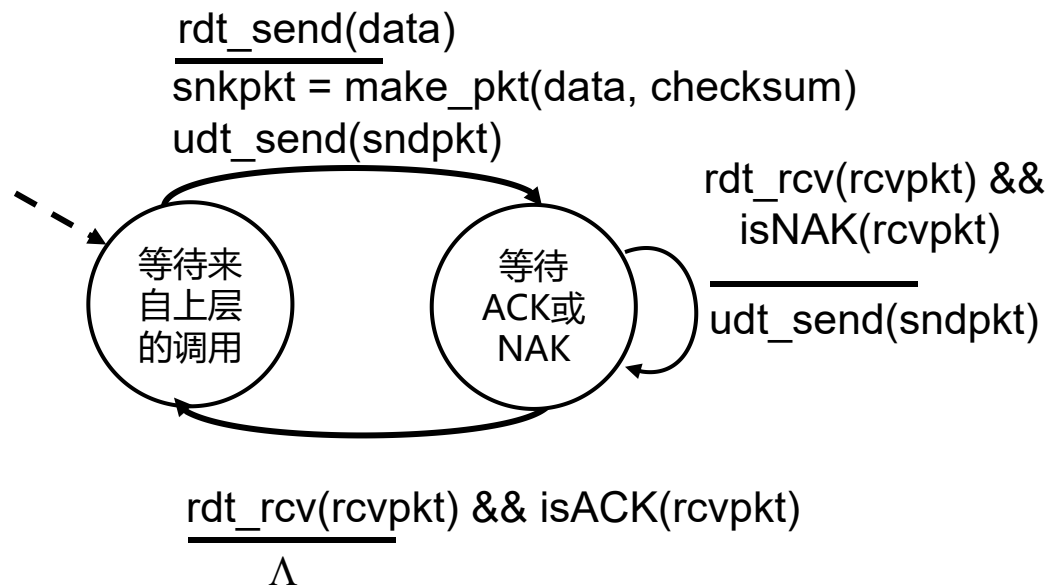
接收方

rdt2.0: 具有比特差错的信道

- 底层的信道可能导致数据包中的**比特差错**（如0和1的翻转）
 - 检测和检测比特差错
- 怎么从错误中恢复
 - **确认 (ACKs)**: 接收方告诉发送方 pkt 正确收到
 - **否认 (NAKs)**: 接收方告诉发送方 pkt 未正确收到
 - 发送方当收到NAK的时候重传 pkt
- rdt2.0 处理比特差错的机制可以归纳为:
 - 差错检测
 - 接收方反馈
 - 重传

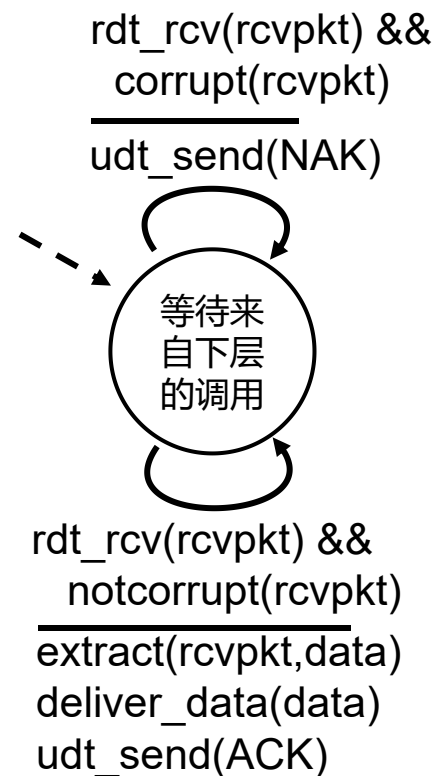


rdt2.0: FSM规格参数

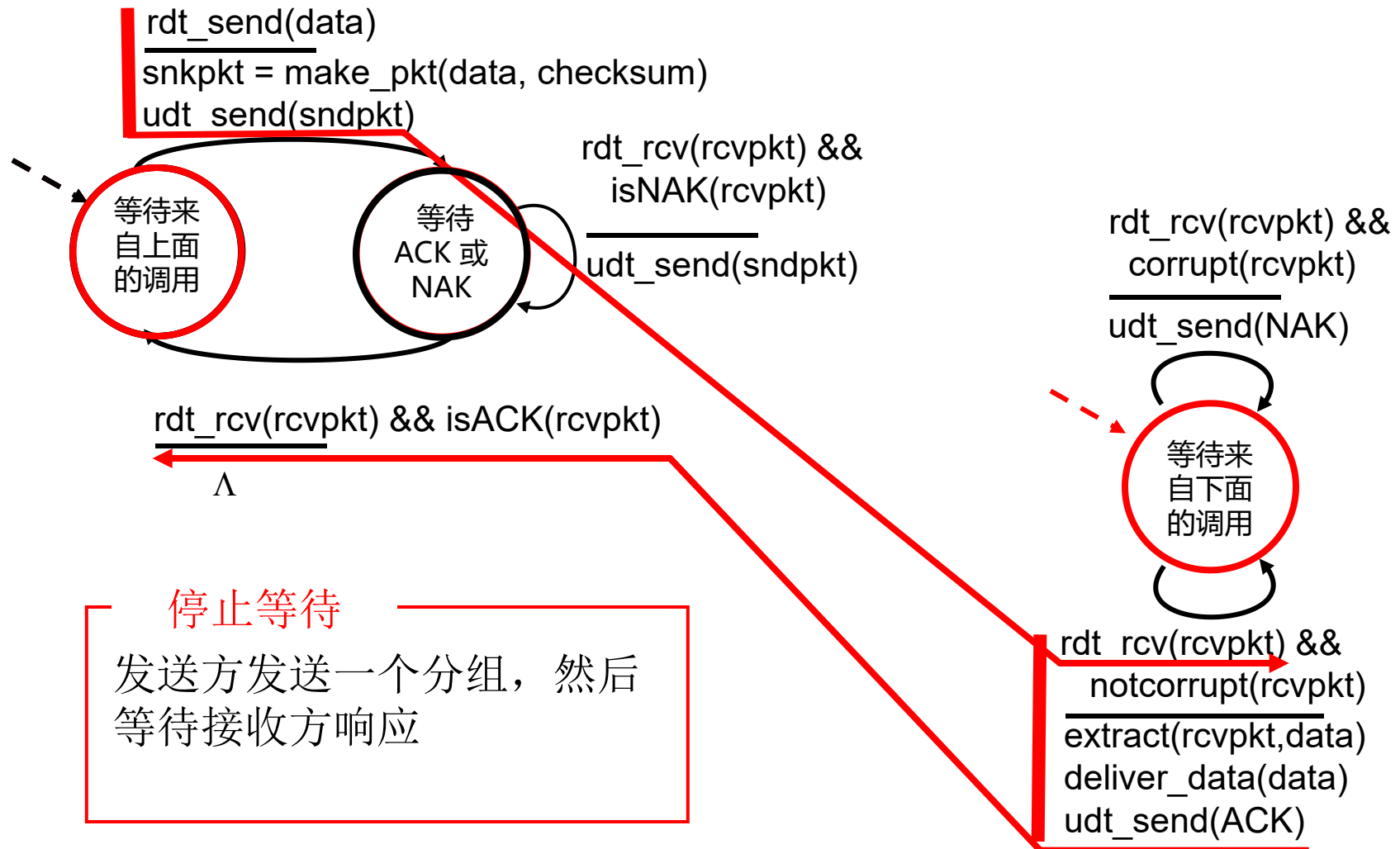


发送方

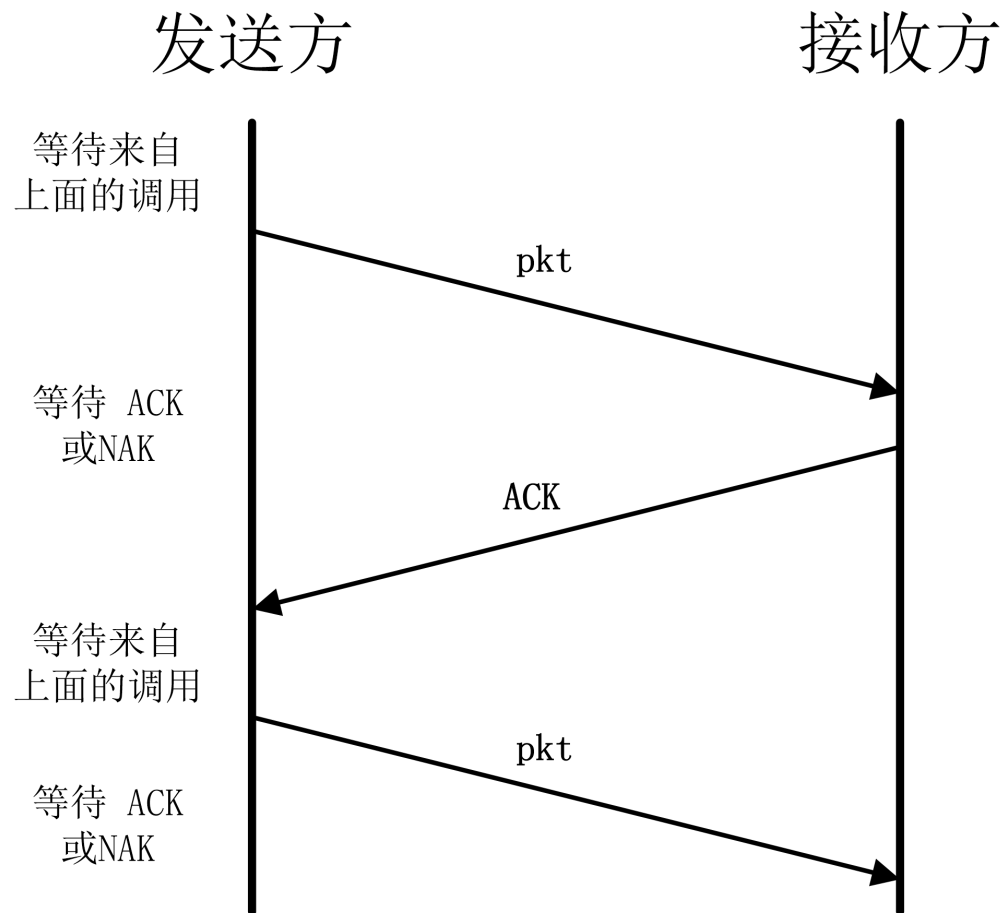
接收方



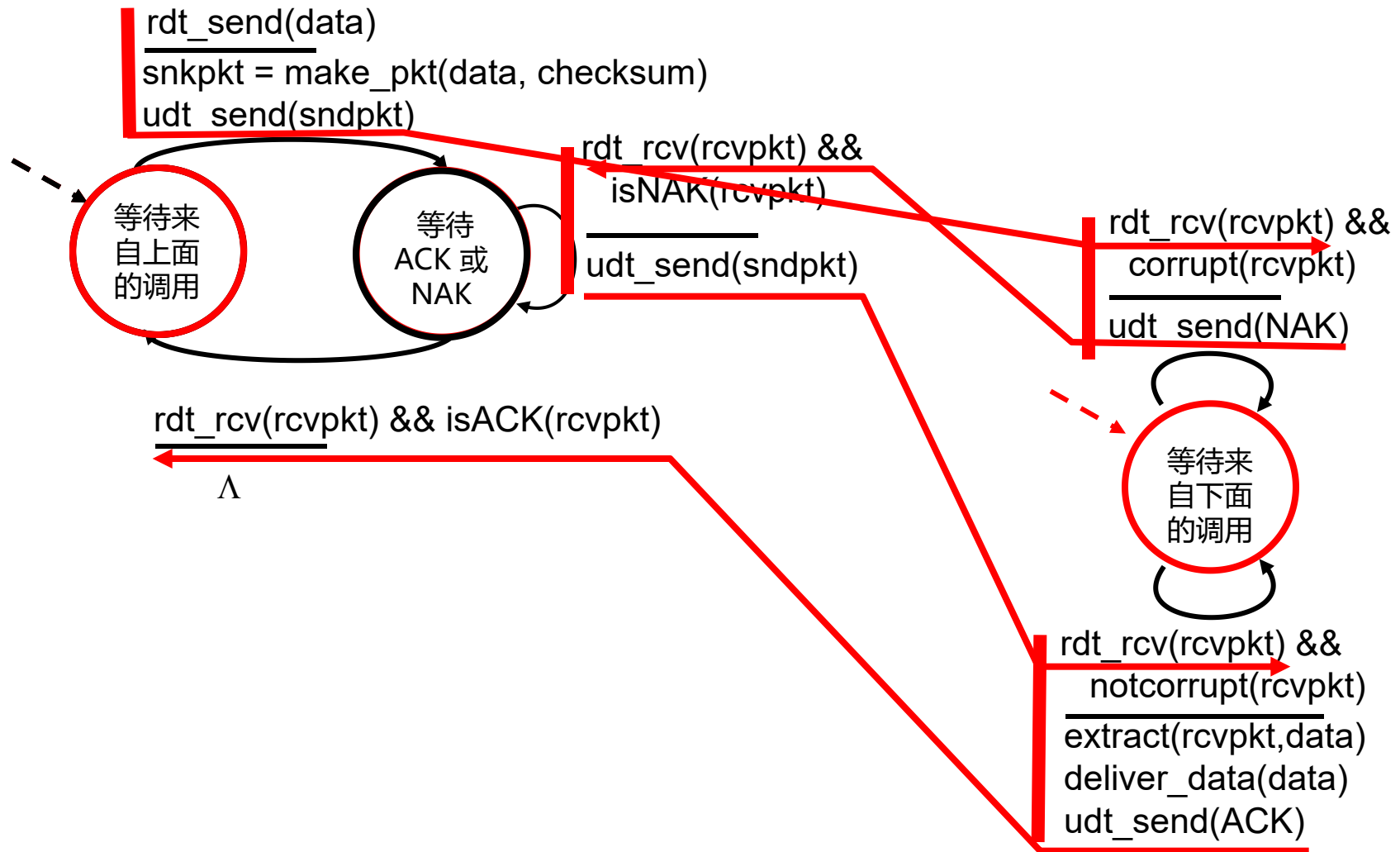
rdt2.0: 无差错时的操作



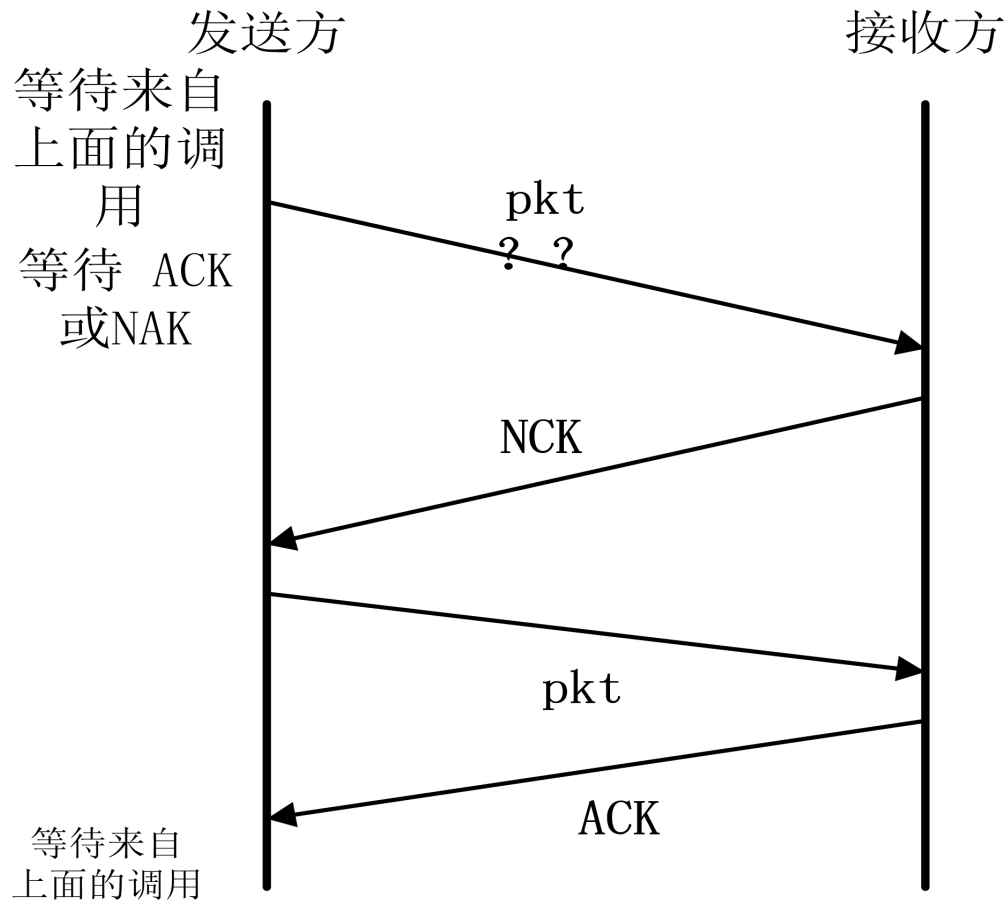
rdt2.0: 无差错时的操作



rdt2.0: 有差错时的情况



rdt2.0: 有差错时的情况

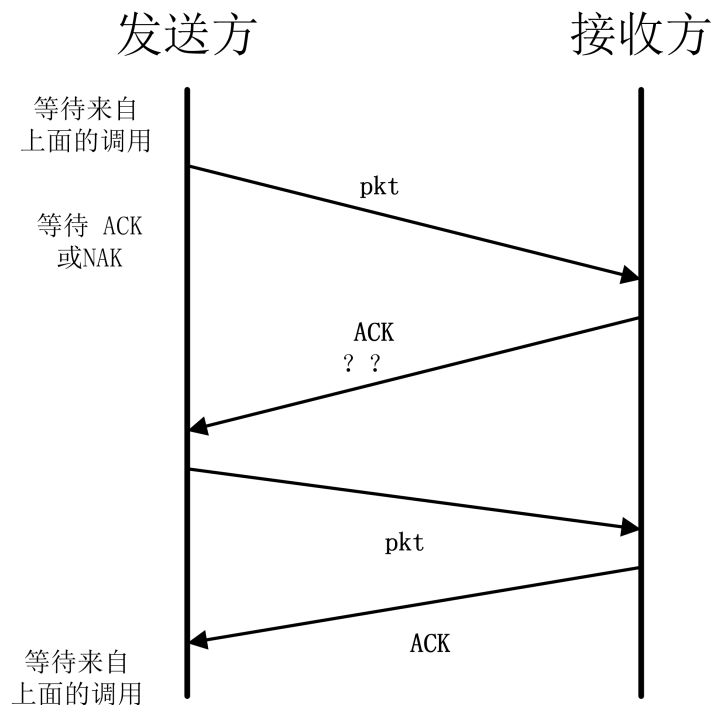
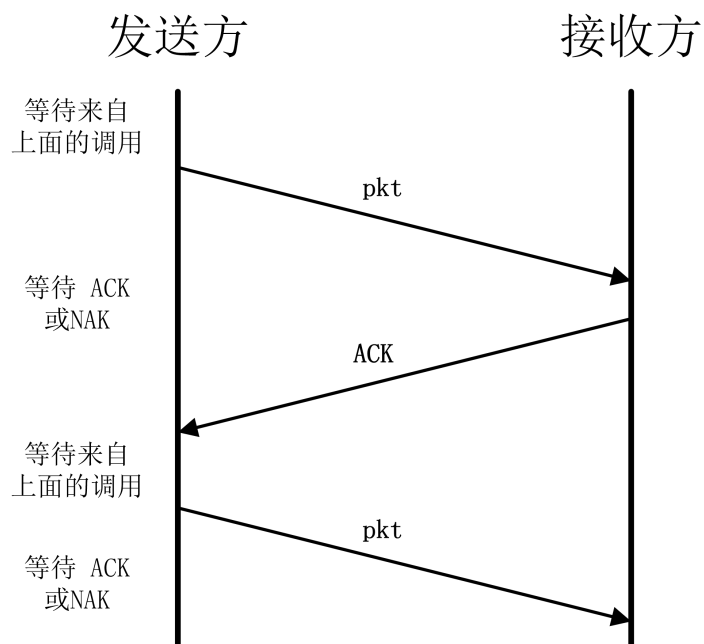


rdt2.0

- rdt2.0似乎可以运行了☺?
- **但是**如果ACK/NAK受损，将会出现何种情况？
 - 发送方不知道在接收方是否正确接收了上个分组
- 重传？
 - 但是不能解决问题，重传会产生冗余分组，接收方不知他的ACK是否被发送方正确收到，所以无法知道收到的是新的分组还是上个分组的重传
- 用**分组序列号**来区分是上个分组的重传还是新的分组
- 处理冗余：
 - 发送方对每个分组增加序列号
 - 如果ACK/NAK受损，发送方重传当前的分组
 - 接收方根据序列号丢弃(不再向上交付)冗余分组



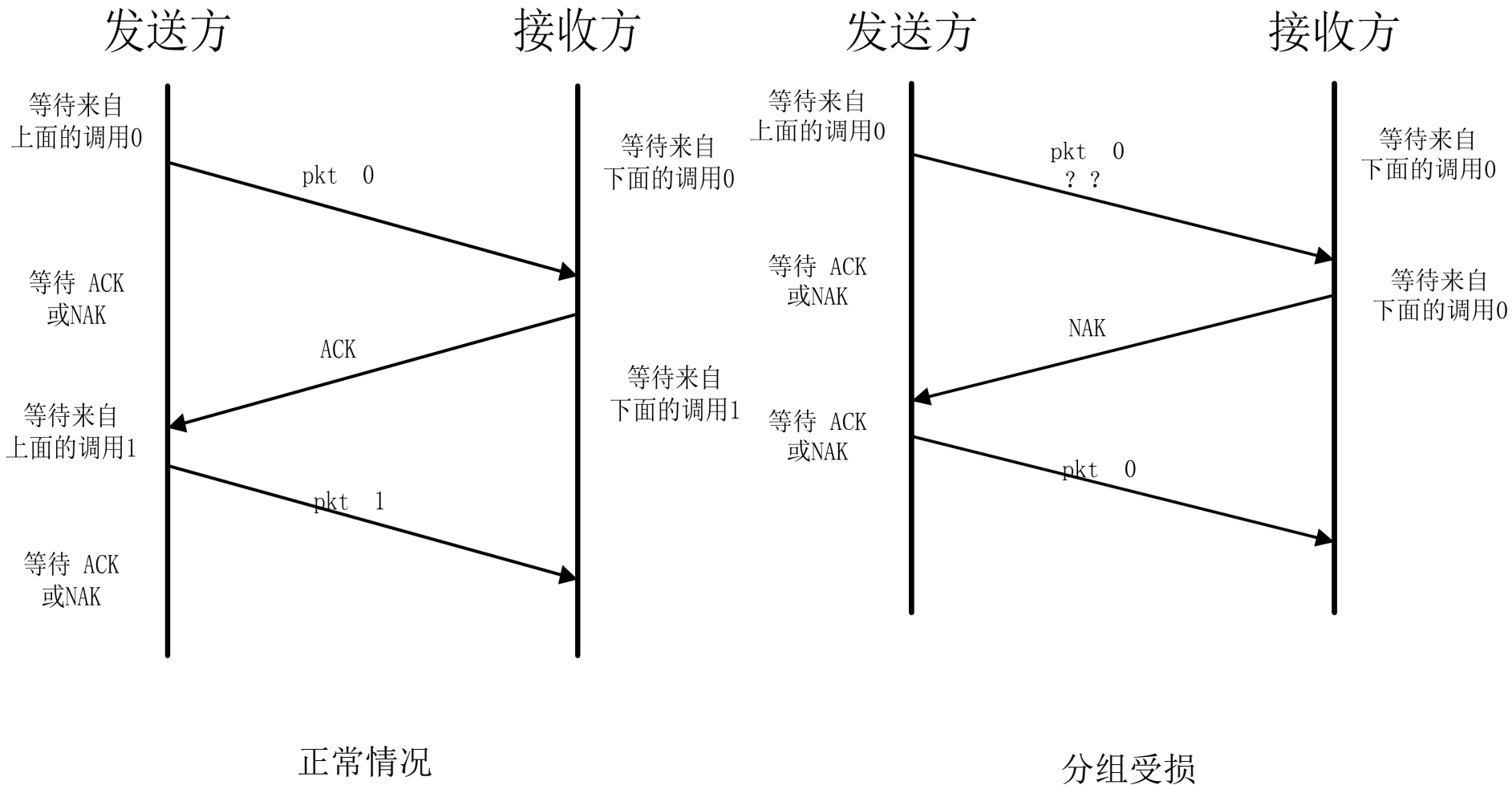
rdt2.0有重大的缺陷!



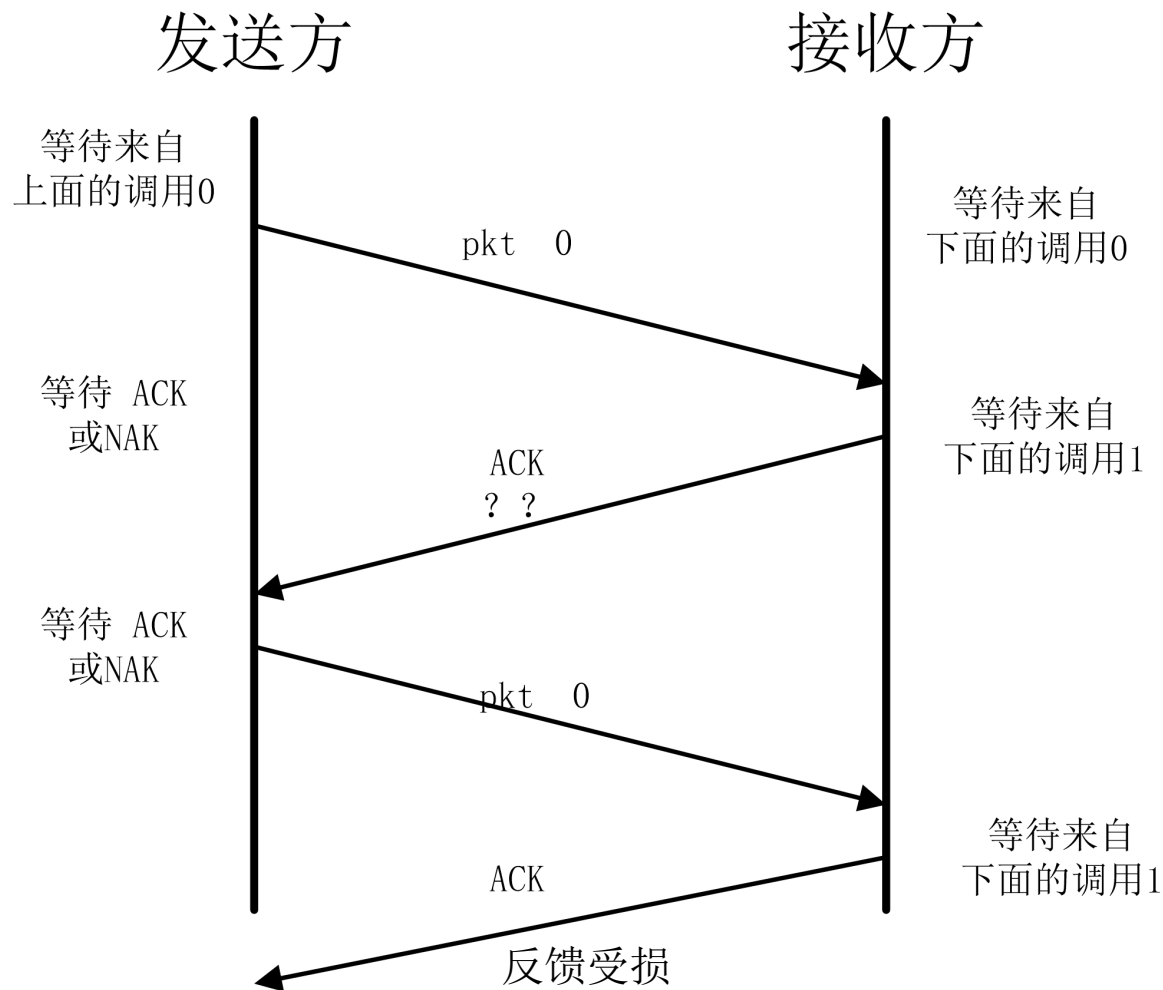
不能只是重传：接收方不知收到的是新的分组还是上个分组的重传



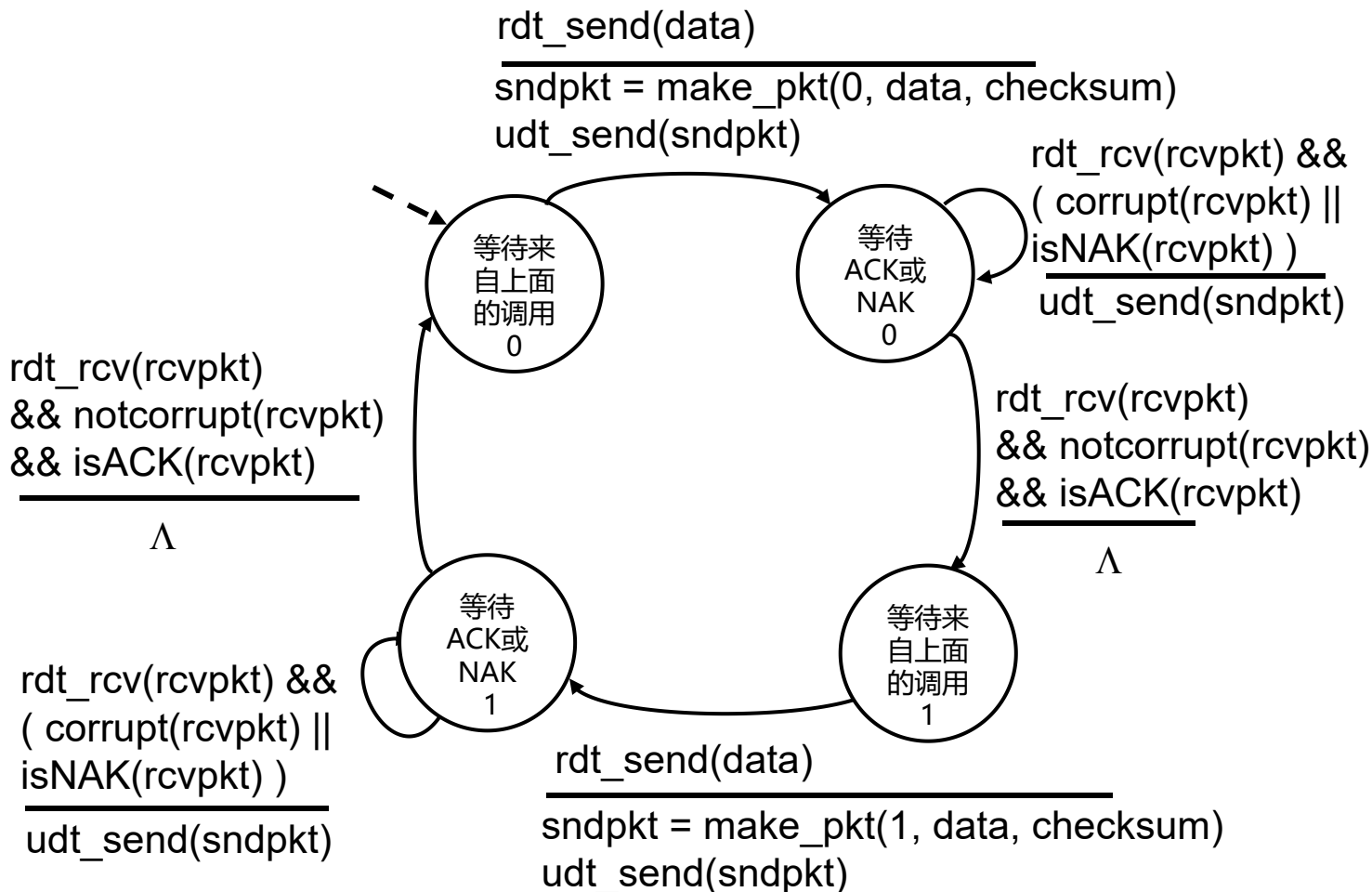
rdt2.1: 发送方处理受损的ACK/NAK



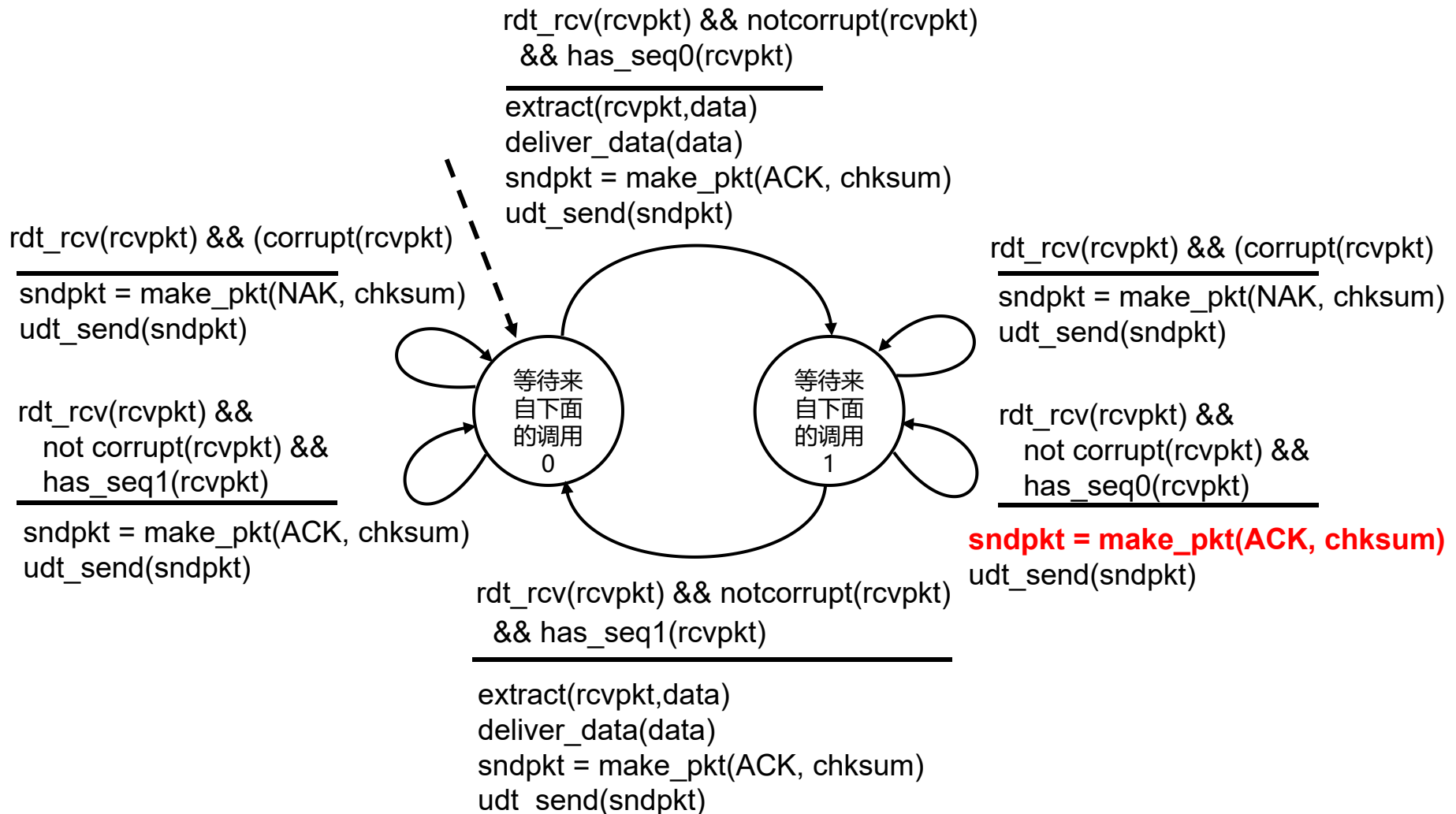
rdt2.1: 发送方, 处理受损的ACK/NAK



rdt2.1: 发送方, 处理受损的ACK/NAK



rdt2.1: 接收方,处理受损的ACK/NAK



rdt2.1: 讨论

■ 发送方:

- 序号seq # 加入分组中
- 两个序号seq. #'s (0,1) 既可以满足条件（为什么？）
- 检查收到的ACK/NAK是否受损
- 状态增加一倍
 - 状态必须“记住”是否“当前的”分组具有0或1序号

■ 思考:

- NAK的必要性？

■ 接收方:

- 必须检查是否接收到的分组是冗余的
 - 状态指示是否0或1是所期待的分组序号seq #

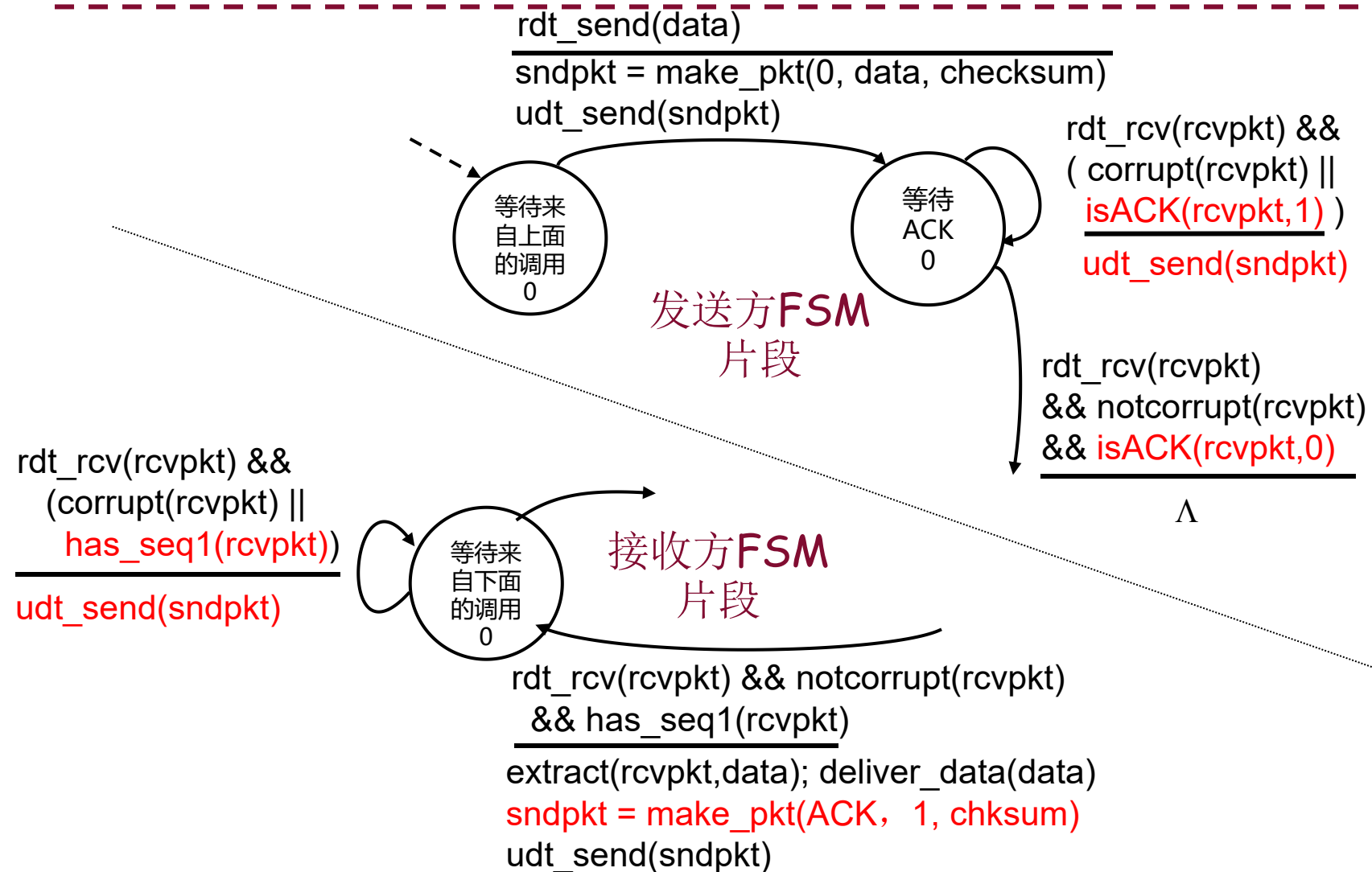


rdt2.2: 一种无NAK的协议

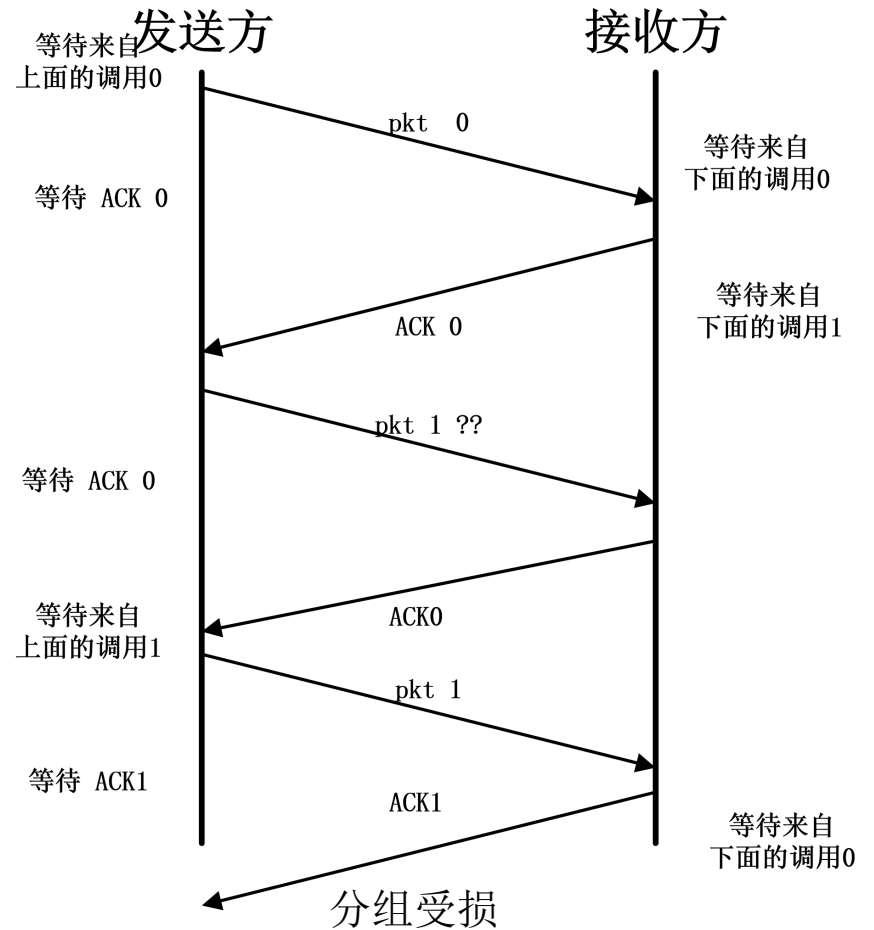
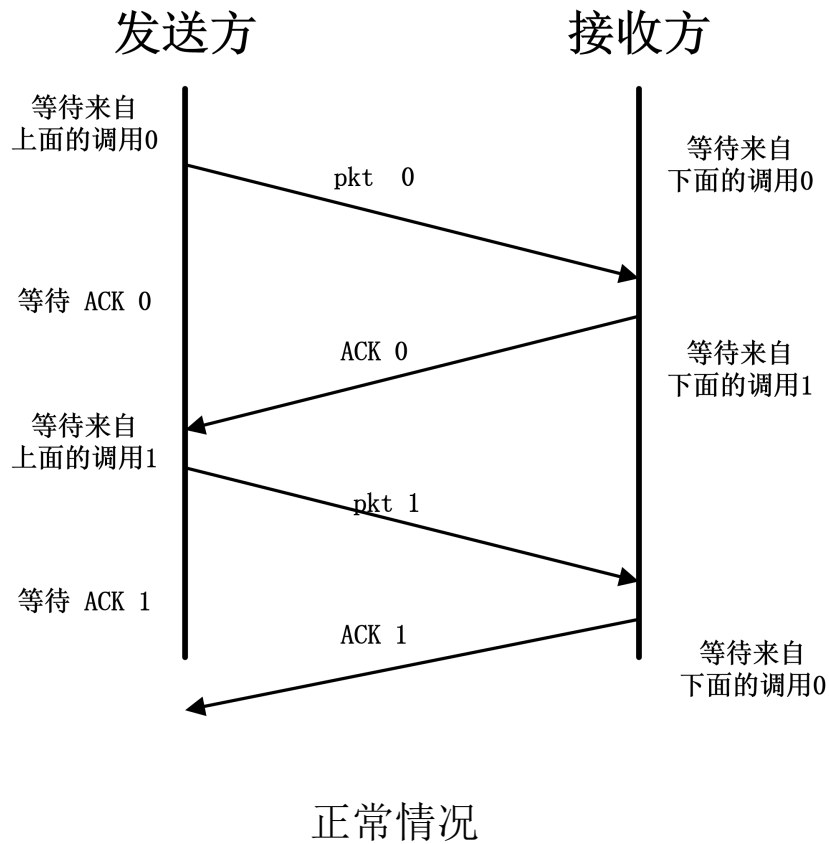
- 与rdt2.1一样的功能，但仅使用ACK
- 代替NAK,接收方对最后正确接收的分组发送ACK
 - 接收方必须明确地包括被确认分组的序号
- 在发送方收到冗余的ACK导致如同NAK相同的动作
 - 重传当前分组
- 在特定的情况下，可以对差错问题进行一定的优化



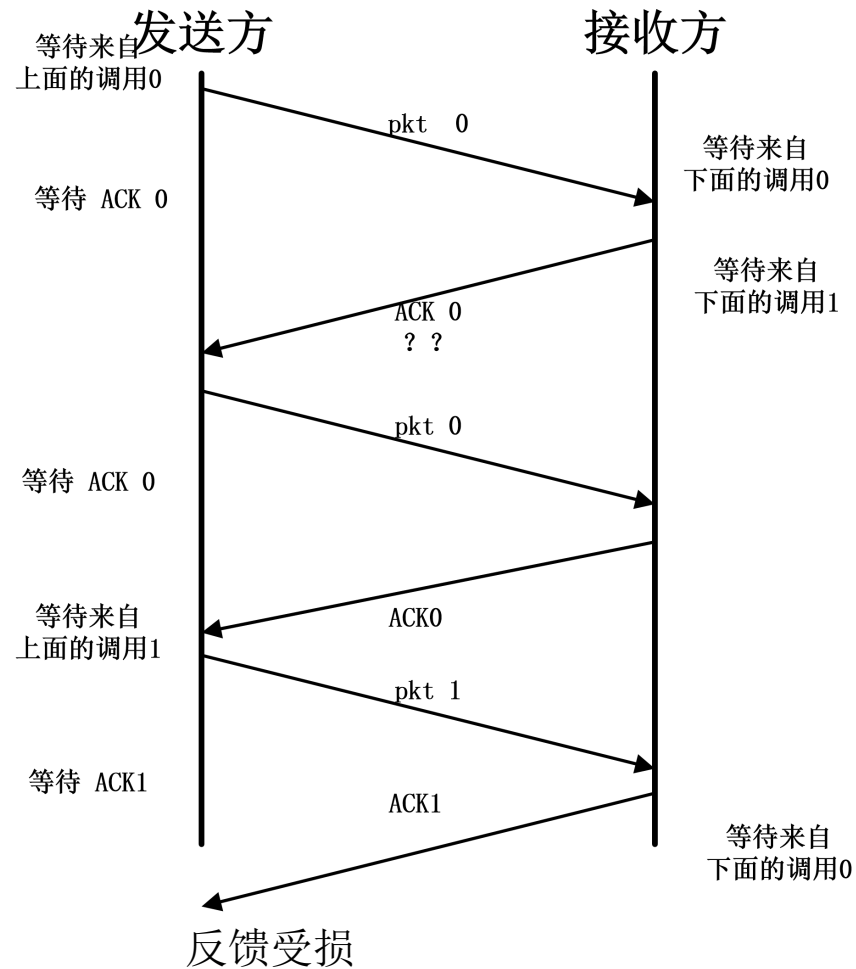
rdt2.2: 发送方, 接收方片段 (区别)



rdt2.2



rdt2.2



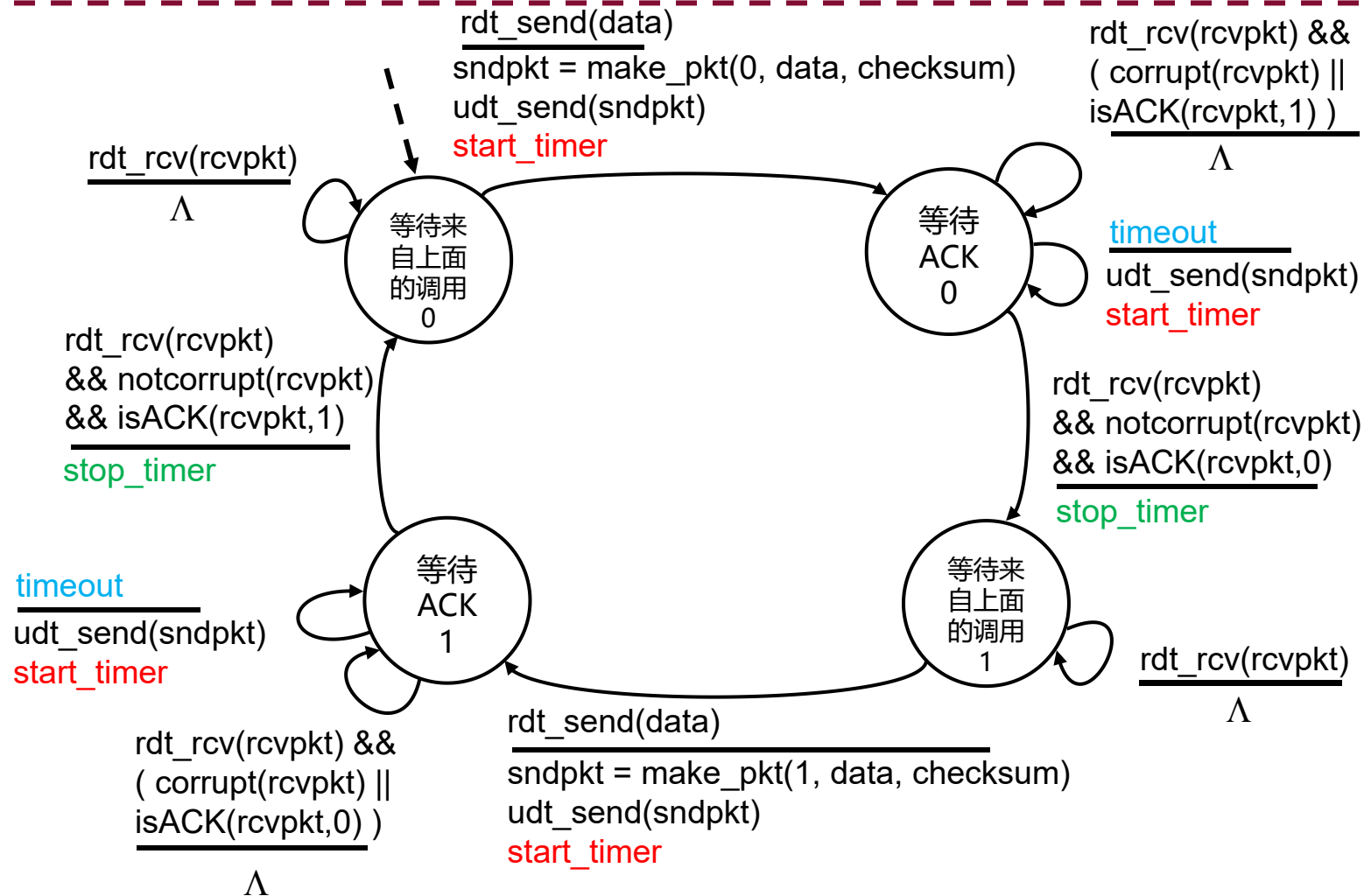
rdt3.0: 具有差错和丢包的信道

- **新假设:**
- 下面的信道也能丢失分组(数据或ACK)
- 协议必须解决两个问题:
 - 怎么检测丢包
 - 以及发生丢包后该做些什么
- 检测丢包有很多方法, 现在我们假设由发送方来解决
- 方法: 发送方等待ACK一段“合理的”时间
- 如在这段时间没有收到ACK则重传
- 如果分组(或ACK)只是延迟(没有丢失):
 - 重传将是冗余的, 但序号的使用已经处理了该情况
- 需要倒计时定时器

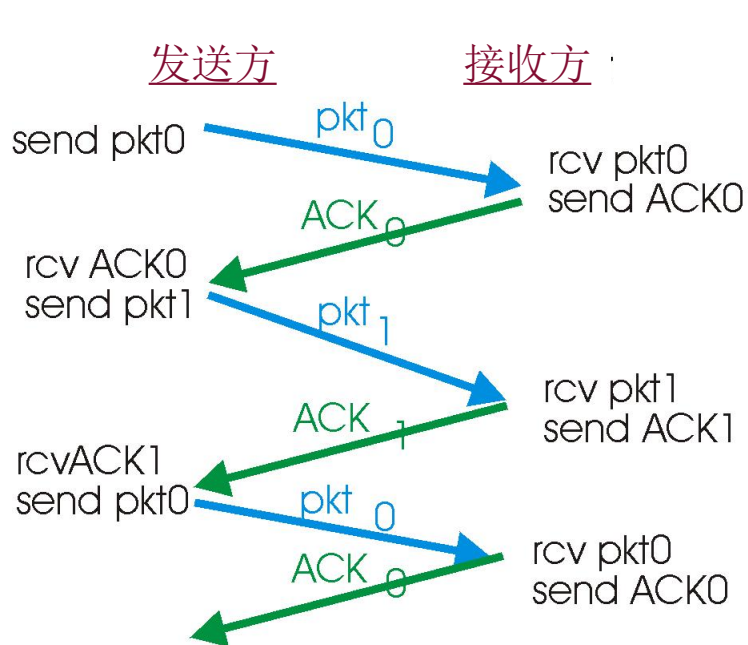


timeout

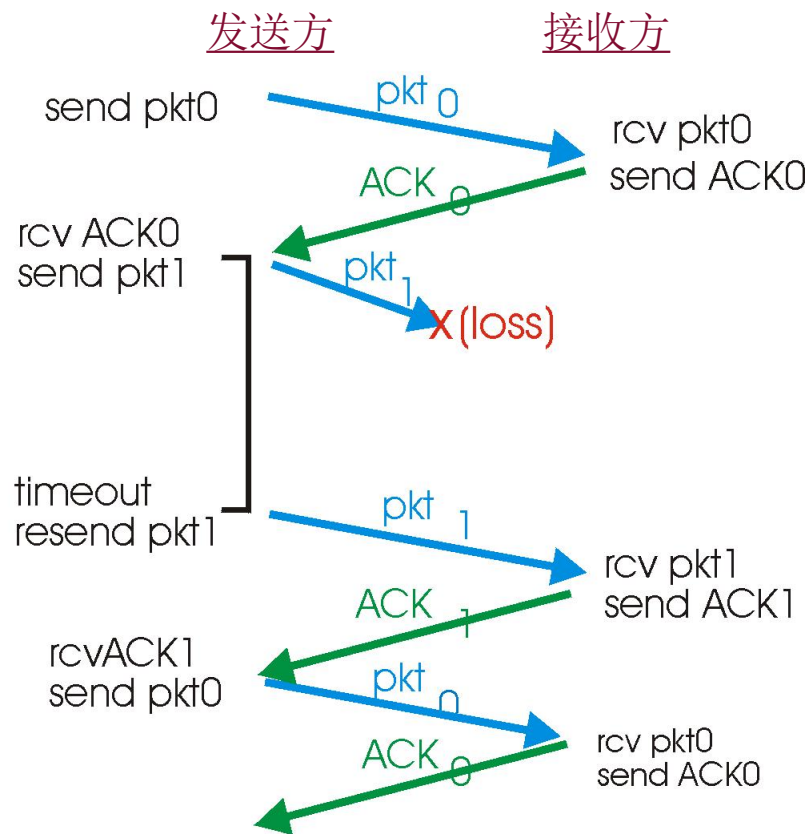
rdt3.0发送方



rdt3.0 运行情况

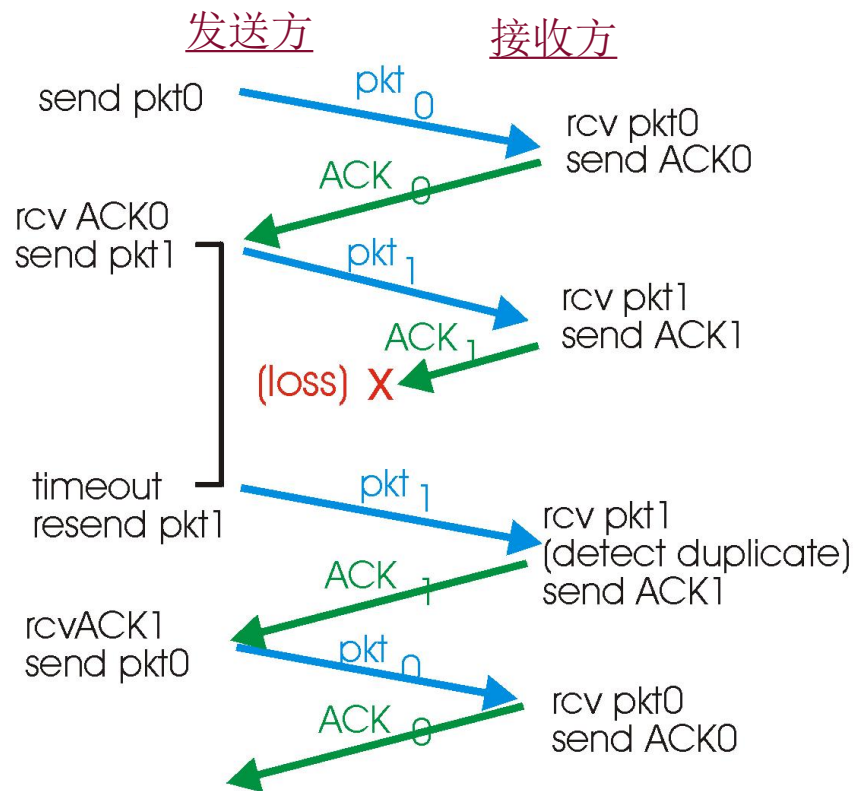


(a) 无丢包时的运行

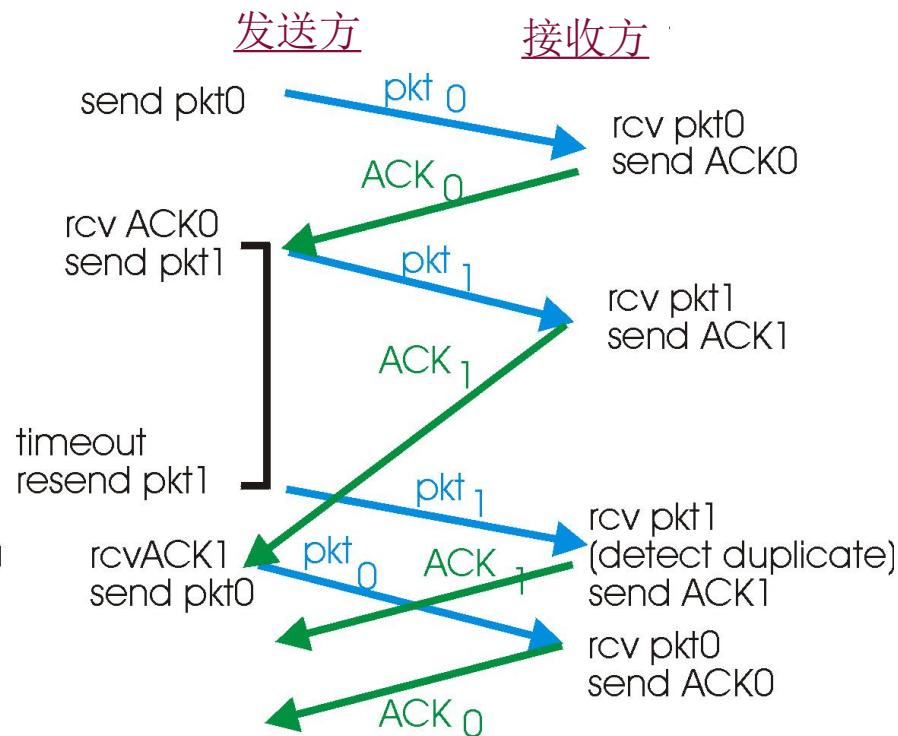


(b) 分组丢失

rdt3.0运行情况

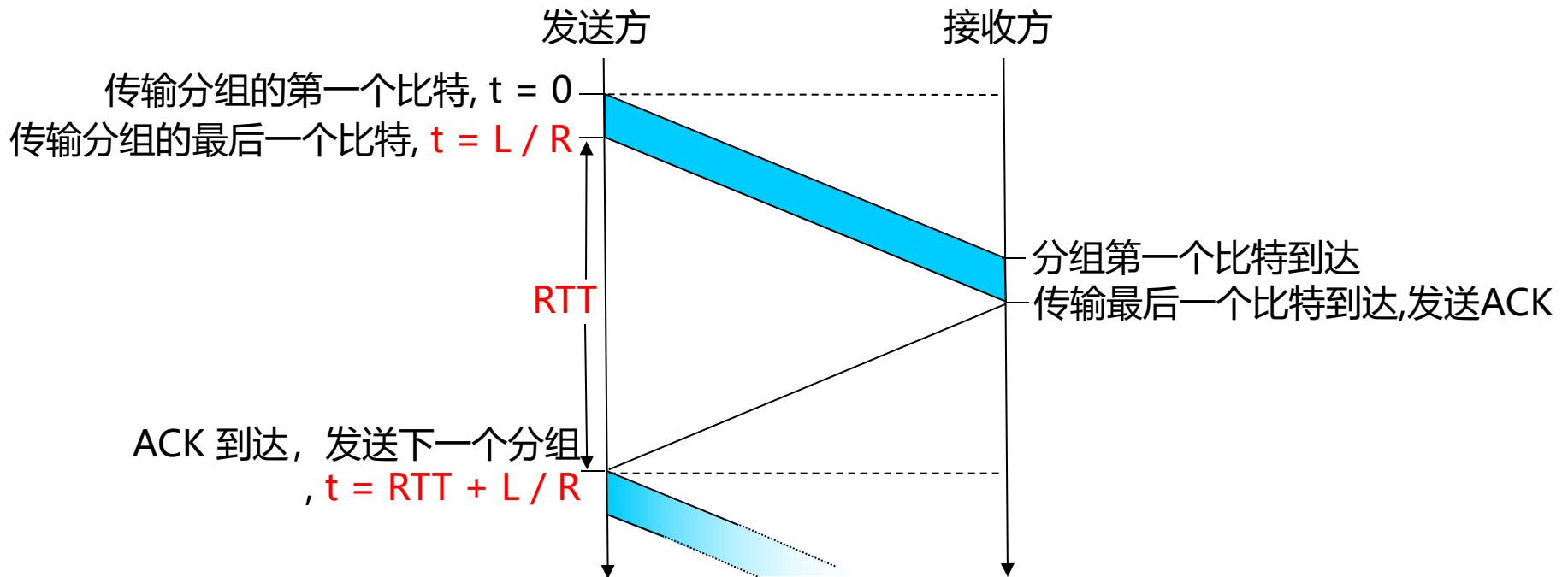


(c) ACK丢失



(d) 过早超时

rdt3.0: 停等协议的运行



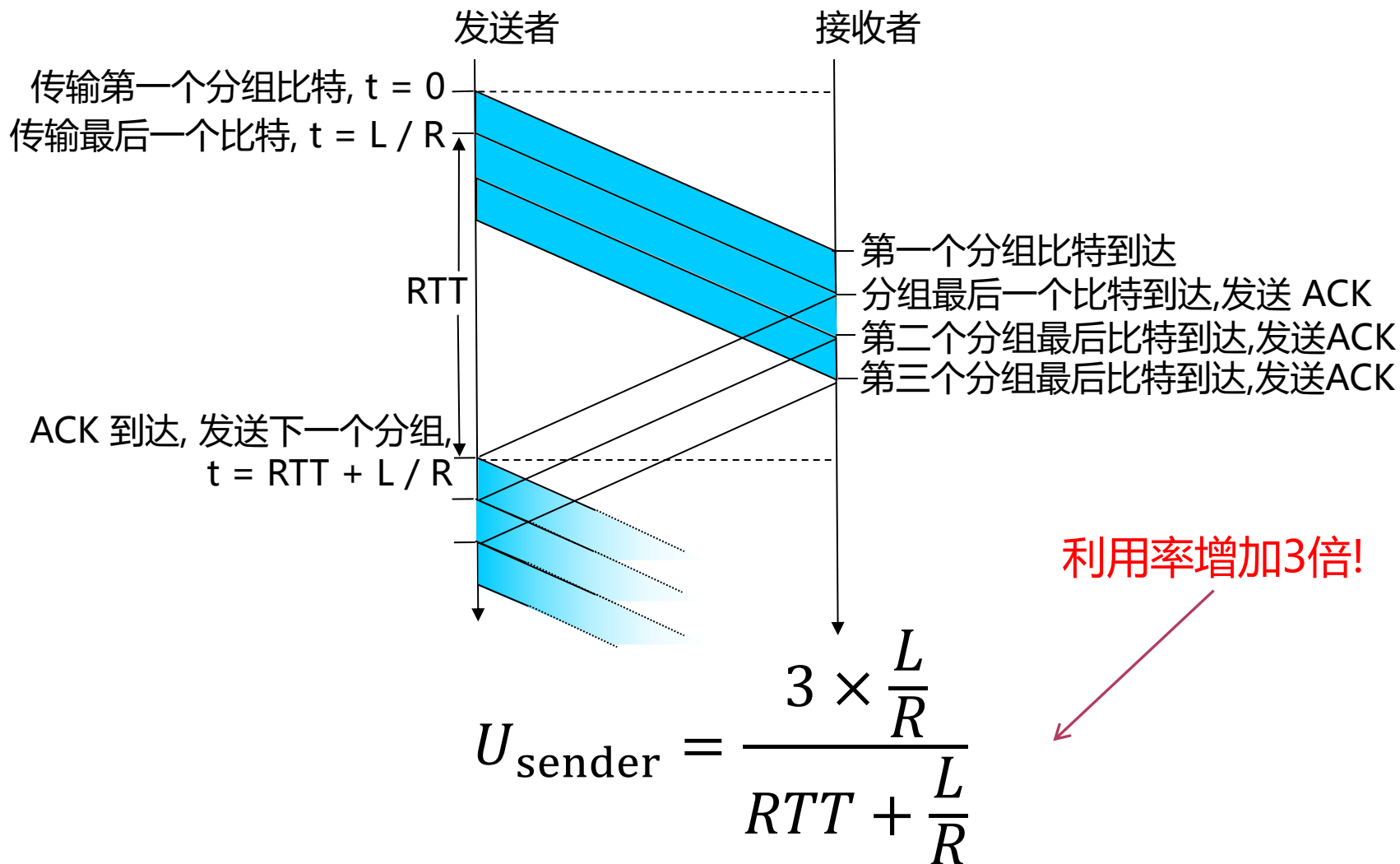
$$U_{\text{sender}} = \frac{\frac{L}{R}}{RTT + \frac{L}{R}}$$

rdt3.0性能问题

- rdt3.0能解决丢失和错误问题，但性能不好
- 例：1Gbps线路，RTT=16ms，分组大小2KB（16000bit）
- 实际发送速率 $T_{transmit} = \frac{L}{R} = \frac{16000}{10^9} = 16 \times 10^{-6} s$
- 利用率： $U_{sender} = \frac{\frac{L}{R}}{RTT + \frac{L}{R}} = \frac{16 \times 10^{-6}}{16 \times 10^{-3} + 16 \times 10^{-6}} = 0.0999\%$
- 实际可使用的时间大部分在等待上



流水线协议: 增加利用率



流水线协议

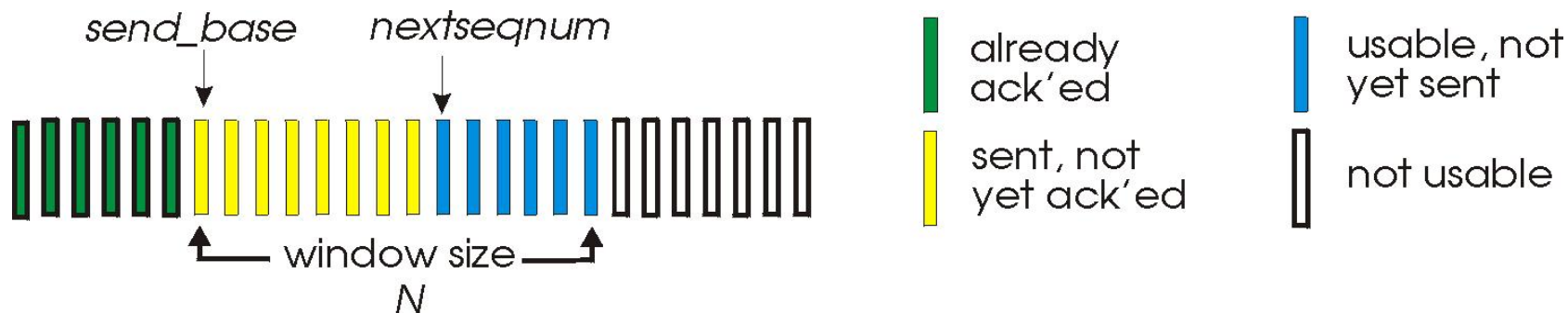
- 流水线: 发送方允许发送多个“传输中的”、还没有应答的报文段
 - 序号的范围必须增加
 - 发送方和/或接收方设有缓冲
- 流水线协议的两种形式:
 - 回退N帧法 (go-Back-N)
 - 选择性重传 (S-R)



Go-Back-N

■ 发送方:

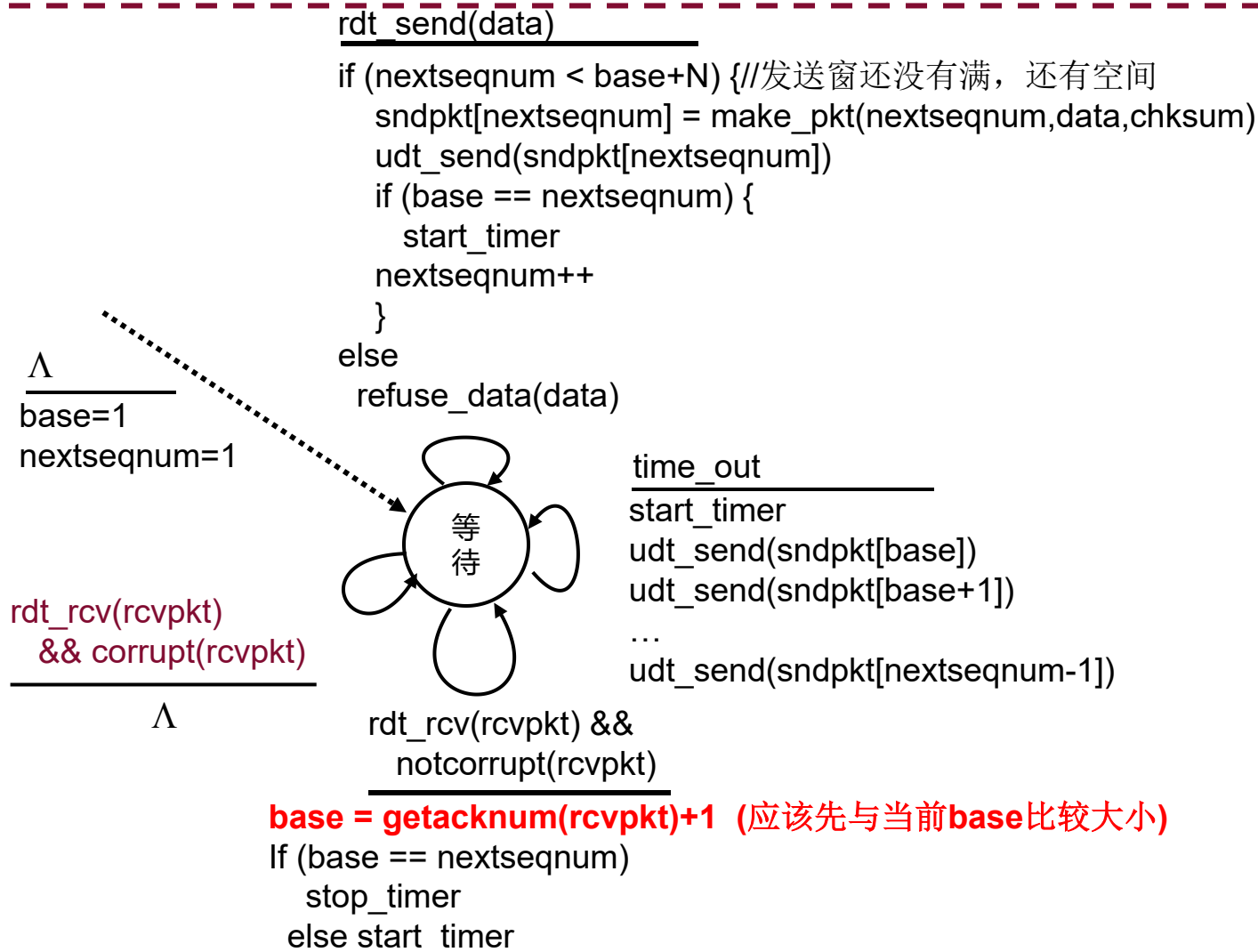
- 在分组首部需要k比特序号, 序号范围是 2^k ,
- “窗口”最大为N, 允许N个连续的没有应答分组



■ 发送方必须响应3类事件

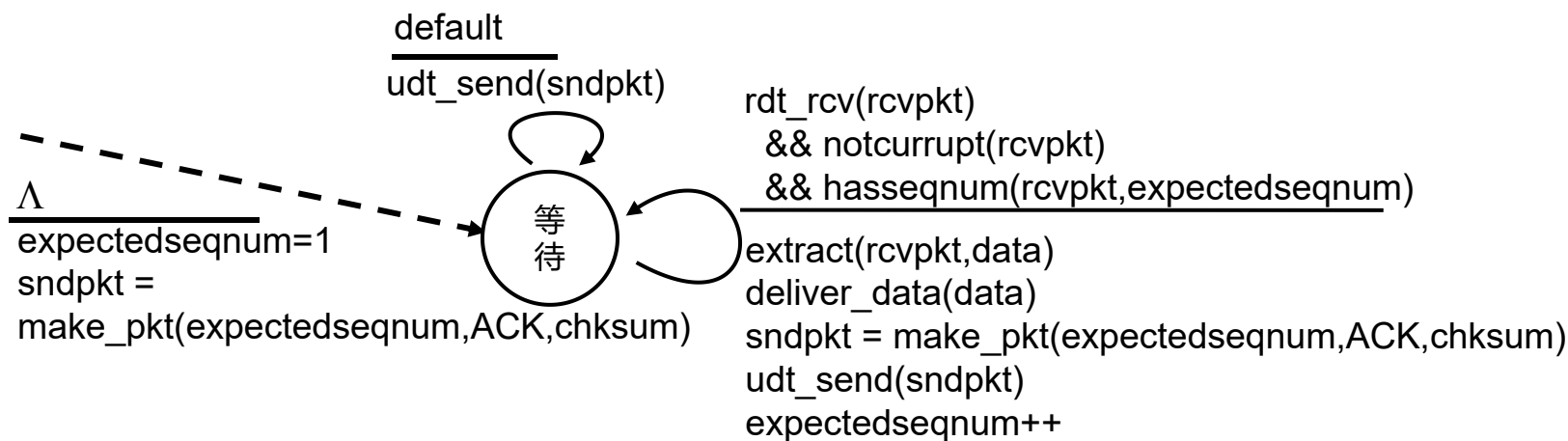
- 上层的调用: 检查发送窗口是否已满
- ACK(n): 确认所有的 (包括序号n) 的分组 - “累计ACK”
- timeout(n): 若超时, 重传窗口中的分组n及所有以前的分组。对每个传输中的分组的用同一个计时器

GBN: 发送方扩展的 FSM

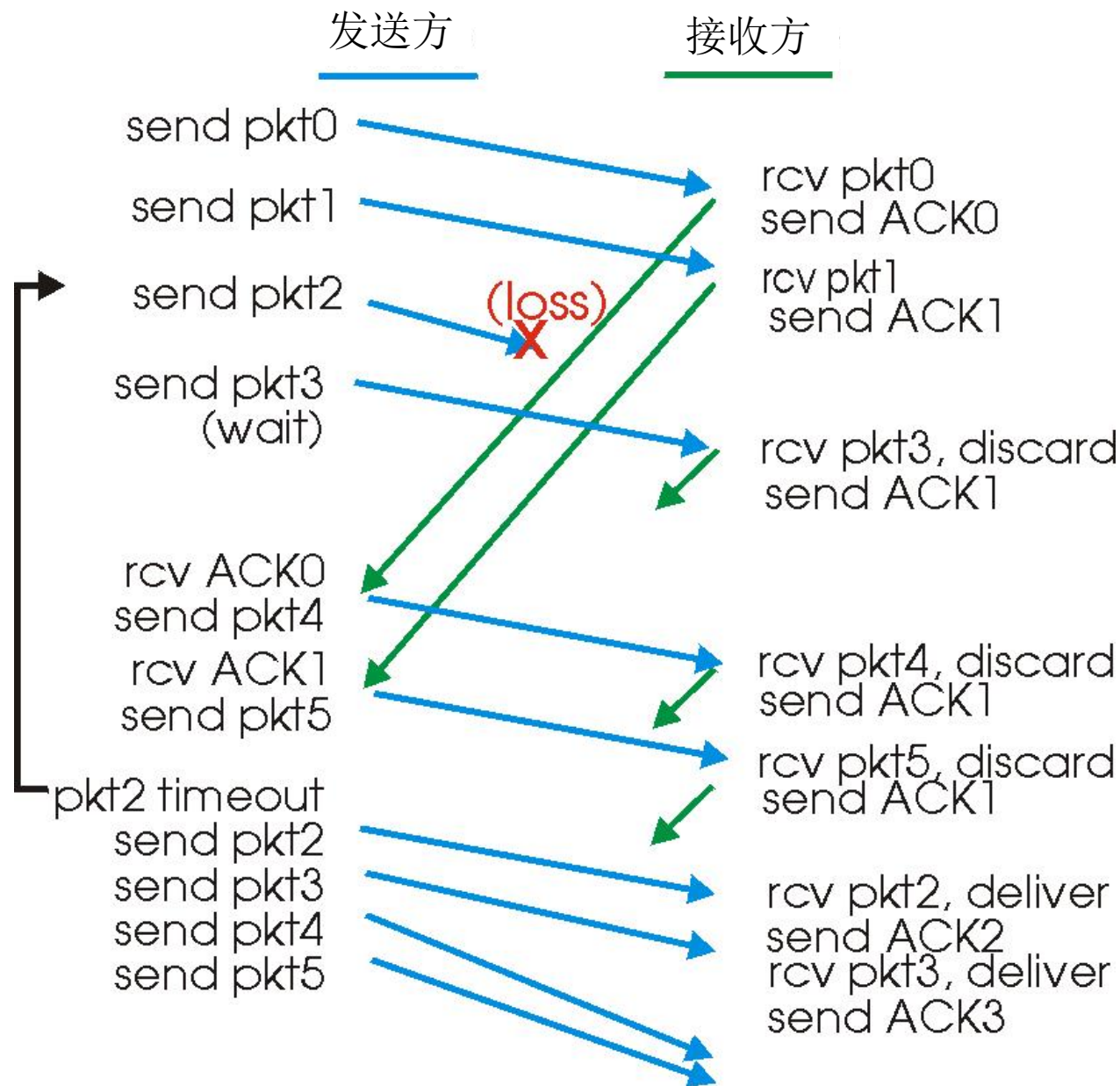


GBN: 接收方扩展 FSM

- 对正确接收的分组总是发送具有最高按序序号的ACK
 - 可能产生冗余的ACKs
 - 仅仅需要记住期望的序号值 (expectedseqnum)
- 对失序的分组:
 - 丢弃 (不缓存) -> 没有接收缓冲区!
 - 用最高的连续序号i重发ACKi报文



GBN 操作

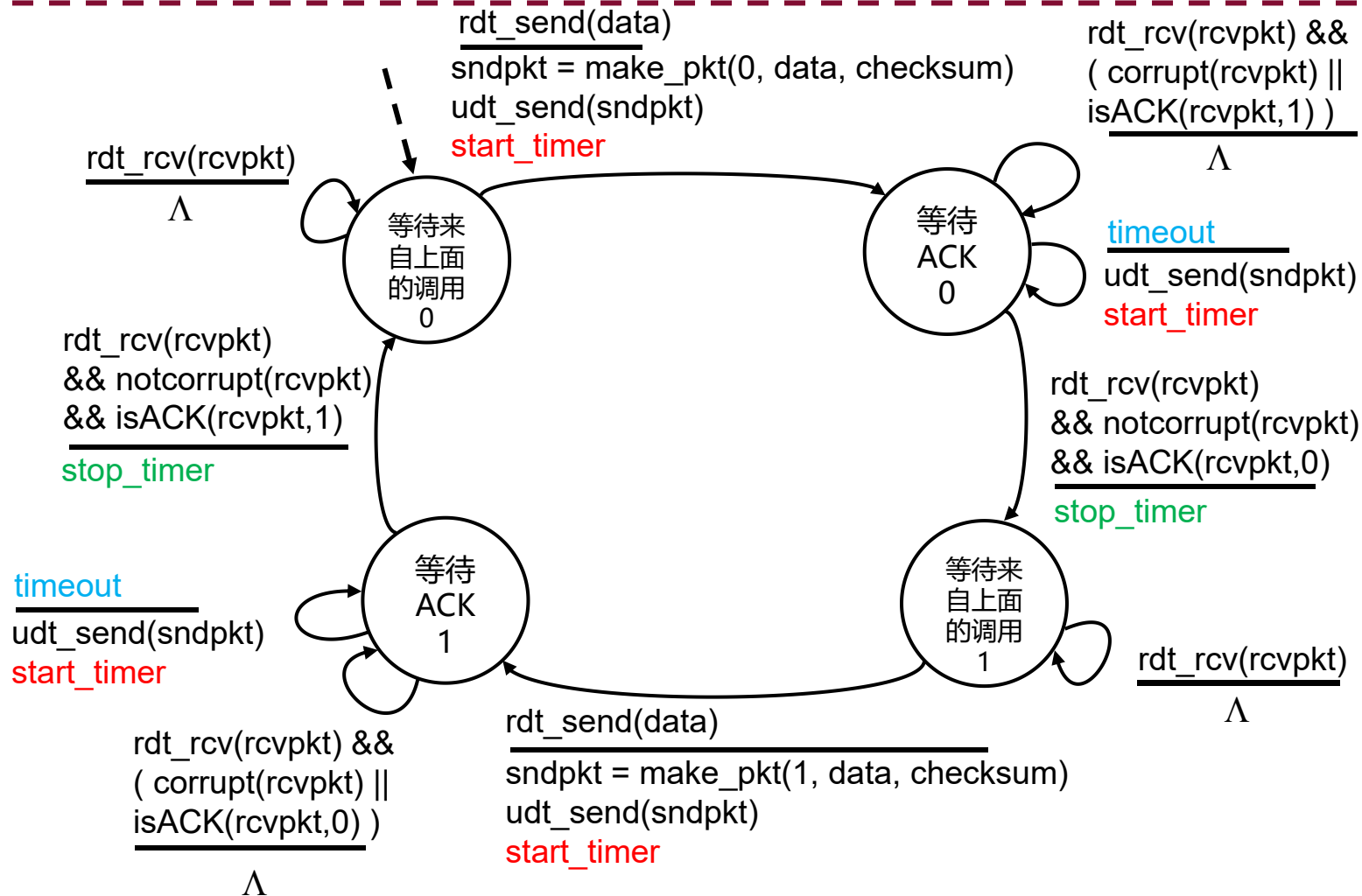


GBN状态

- GBN只有一个状态
- 与rdt3.0的区别?
 - 引入了实际的序号
- 为什么可以同时实现差错、丢失、乱序?
 - 在引入序号的条件下，多个状态是如何合并的？
 - 合并之后有什么影响？



回顾rdt3.0发送方状态

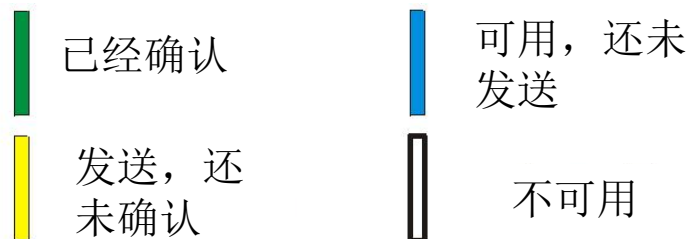
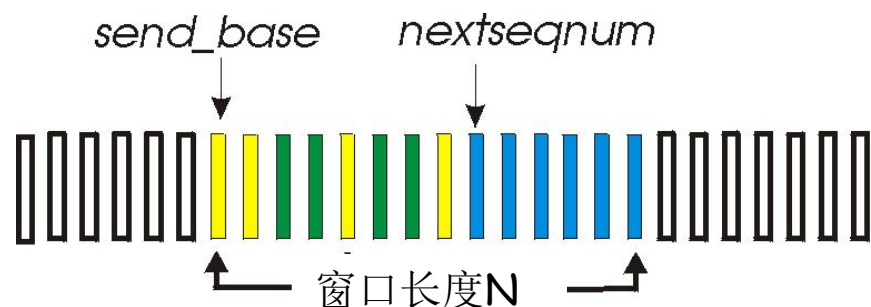


选择性重传 (Selective Repeat)

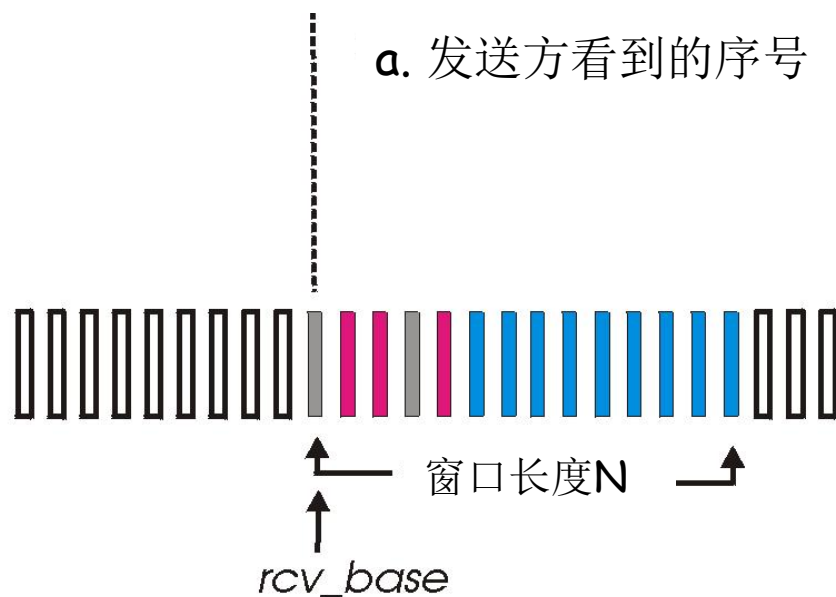
- GBN改善了信道效率, 但仍然有**不必要重传**问题
- 选择性重传:
 - 接收方分别确认所有正确接收的报文段
 - 需要缓存报文段, 以便最后按序交付给上层
 - 发送方只需要重传没有收到ACK的报文段
 - 发送方对每个没有确认报文段有一个单独的计时器
- 发送窗口(同GBN)
 - 也需要限制已发送但尚未应答报文段的序号



选择性重传: 发送方, 接收方窗口



a. 发送方看到的序号



b. 接收方看到的序号

选择性重传

发送方

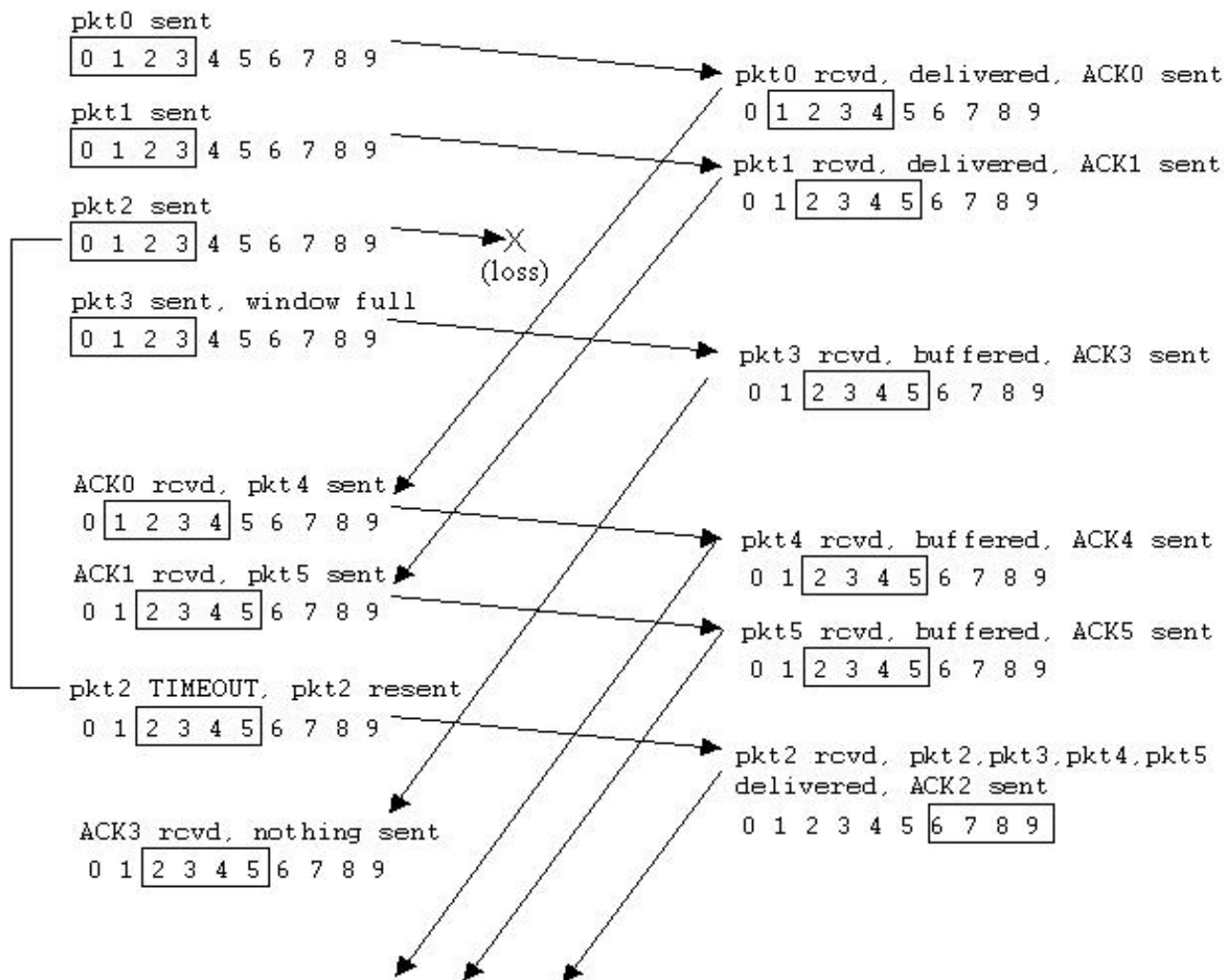
- 上层传来数据：
 - 如果窗口中下一个序号可用, 发送报文段
- timeout(n):
 - 重传分组n, 重启其计时器
- ACK(n) 在[sendbase,sendbase+N]:
 - 标记分组 n 已经收到
 - 如果n 是最小未收到应答的分组, 向前滑动窗口base指针到下一个未确认序号

接收方

- 分组n在 [rcvbase, rcvbase+N-1]
 - 发送 ACK(n)
 - 失序: 缓存
 - 按序: 交付 (也交付所有缓存的按序分组), 向前滑动窗口到下一个未收到报文段的序号
- 分组n在[rcvbase-N,rcvbase-1]
 - ACK(n)
- 其他:
 - 忽略

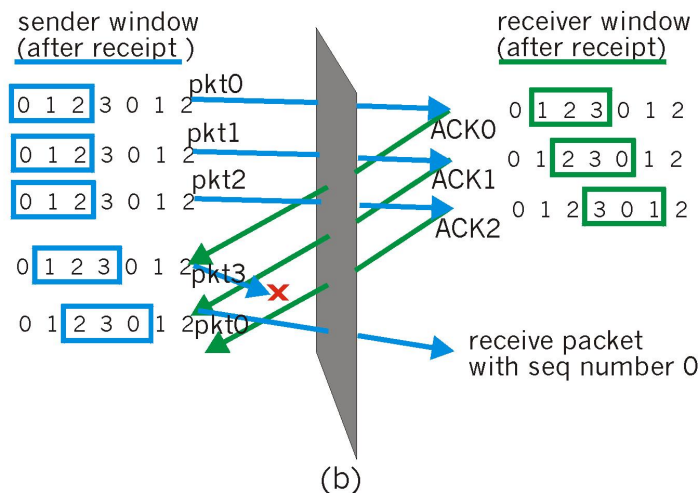
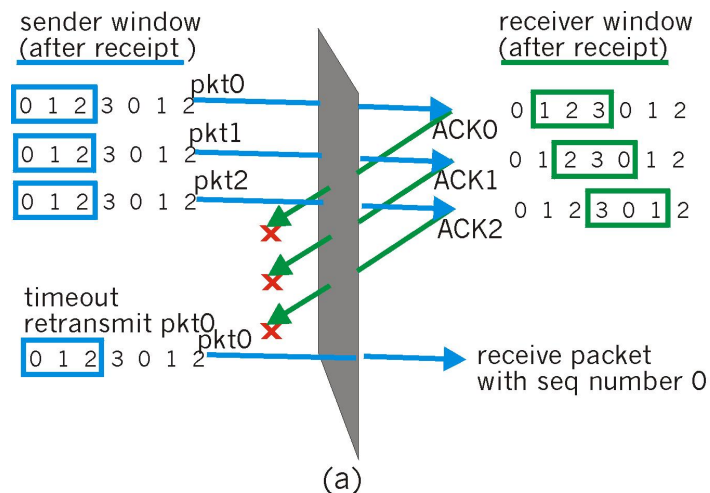


选择重传的操作



选择重传: 困难的问题

- 例子: 难以区分新分组还是重传
 - 序号: 0, 1, 2, 3
 - 窗口长度 = 3
- 接收方: 在(a)和(b)两种情况下接收方没有发现差别!
 - 在(a)中不正确地将冗余的当为新的, 而在(b)中不正确地将新的当作冗余的
- 问题: 序号长度与窗口长度有什么关系?
 - 回答: 窗口长度小于等于序号空间的一半



可靠数据传输机制及用途总结

机制	用途和说明
检验和	用于检测在一个传输分组中的比特错误。
定时器	用于检测超时/重传一个分组 ，可能因为该分组（或其ACK）在信道中丢失了。由于当一个分组被时延但未丢失（过早超时），或当一个分组已被接收方收到但从接收方到发送方的ACK丢失时，可能产生超时事件，所以接收方可能会收到一个分组的多个冗余拷贝。
序号	区分不同的分组 ，用于为从发送方流向接收方的数据分组按顺序编号。所接收分组的序号间的空隙可使该接收方检测出丢失的分组。具有相同序号的分组可使接收方检测出一个分组的冗余拷贝。
确认	接收方 用于告诉发送方一个分组或一组分组已被正确地接收到了 。确认报文通常携带着被确认的分组或多个分组的序号。确认可以是逐个的或累积的，这取决于协议。
否定确认	接收方用于告诉发送方某个分组未被正确地接收 。否定确认报文通常携带着未被正确接收的分组的序号。
窗口、流水线	发送方也许被限制仅发送那些序号落在一个指定范围内的分组。通过 允许一次发送多个分组但未被确认 ，发送方的利用率可在停等操作模式的基础上得到增加。我们很快将会看到，窗口长度可根据接收方接收和缓存报文的能力或网络中的拥塞程度，或两者情况来进行设置。



基于NAK的可靠传输

- 可以用于无连接状态下的可靠传输
 - 不适用ACK确认数据包
- 接收方在收到数据包时，如果发现有错误或丢失，发送 NAK 给发送方
- 发送方接收到 NAK 后会重新发送相应的数据包
- 思考一下：
 - 在网络传输上有没有优势？什么条件？
 - 之前出现的问题：差错、丢失、乱序需要怎样解决？
 - 有什么其他方面的优势和弊端？
- 实验测试



思考问题

1. rdt3.0的接收方状态是怎样的？
2. rdt3.0的接收方状态在有序号条件下是如何合并成GBN的？
3. 基于NAK的可靠传输的FSM？

